

Unit 4 Practice Test

Practice 4.2

Sam has read that many programmers get paid by the number of lines of code they generate. Wanting to maximize the size of his paycheck, he has decided to write a program to count the number of lines in a given file.

Problem

Write a program to perform the following tasks:

1. Attempt to gather the filename from the command line
2. If no filename is present, prompt the user for the filename
3. Loop through the file, counting the number of lines
4. Display the results using the correct tense for empty, 1, and more than one.

Example

The user may specify the filename from the command line. Assume the file “example1.txt” has 2 lines:

```
a.out example1.txt  
example1.txt has 2 lines.
```

If no file is specified on the command line, then the program will prompt the user for a file. Assume the file “example2.txt” has 1 line of text:

```
a.out  
Please enter the file name: example2.txt  
example2.txt has 1 line.
```

Note how “line” is singular. Finally, if the file is empty or non-existent, then the following message will appear:

```
a.out missing.txt  
missing.txt is empty.
```

Grading

The test bed is:

```
testBed cs124/practice42 practice42.cpp
```

The sample solution is:

```
/home/cs124/tests/practice42.cpp
```

Continued on the next page

Grading for Test4

Sample grading criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Modularization 10%	Perfect cohesion and coupling	No bugs exist in calling functions, though modularization is not perfect	Redundant data passed between functions	A bug exists passing parameters or calling a function	Only one function used
Command line 10%	Input logic "elegant"	Able to access the filename from the command line and from a prompt	One bug	An attempt was made to accept input from both the command line and from prompts.	No attempt was made to access the command line parameters
File 20%	"Good" and perfect error detection	Able to fetch all the text from a file	One bug	Elements of the solution exist	No attempt was made to read from the file
Problem Solving 30%	The most elegant and efficient solution was found	Zero test bed errors	Correct on at least one test case	Elements of the solution are present	No attempt was made to count all the words <u>or</u> program failed to compile
Handle tense on output 10%	The most elegant and efficient solution was found	All three cases handled	One bug	Logic exists to attempt to detect need for different tenses	No attempt was made to handle tense
Programming Style 20%	Well commented, meaningful variable names, effective use of blank lines	Zero style checker errors	One or two minor style checker errors	Code is readable, but serious style infractions	No evidence of the principles of "elements of style" in the program

Unit 4 Project : Sudoku

Write a program to allow the user to play Sudoku. For details on the rules of the game, see:

<http://en.wikipedia.org/wiki/Sudoku>

The program will prompt the user for the filename of the game he or she is currently working on and display the board on the screen. The user will then be allowed to interact with the game by selecting which square he or she wishes to change. While the program will not solve the game for the user, it will ensure that the user has not selected an invalid number. If the user types 's' in the prompt, then the program will show the user the possible valid numbers for a given square. When the user is finished, then the program will save the board to a given filename and exit.

This project will be done in three phases:

- Project 11 : Design the Sudoku program
- Project 12 : Allow the user to interact with the Sudoku game
- Project 13 : Enforce the rules of the Sudoku game

Interface Design

Consider a game saved as `myGame.txt`:

```
7 2 3 0 0 1 5 9
6 0 0 3 0 2 0 0 8
8 0 0 0 1 0 0 0 2
0 7 0 6 5 4 0 2 0
0 0 4 2 0 7 3 0 0
0 5 0 9 3 1 0 4 0
5 0 0 0 7 0 0 0 3
4 0 0 1 0 3 0 0 6
9 3 2 0 0 0 7 1 4
```

Note that '0' corresponds to an unknown value. The following is an example run of the program. Please see the following program for an example of how this works:

```
/home/cs124/projects/prj13.out
```

Program starts

An example of input is underlined.

```
Where is your board located? myGame.txt
```

With the filename specified, the program will display a menu of options:

```
Options:
? Show these instructions
D Display the board
E Edit one square
S Show the possible values for a square
Q Save and quit
```

After this, the board as read from the file will be displayed:

	A	B	C	D	E	F	G	H	I
1	7	2	3				1	5	9
2	6			3	2				8
3	8				1				2
-----+-----+-----									
4		7		6	5	4			2
5			4	2		7	3		
6		5		9	3	1			4
-----+-----+-----									
7	5				7				3
8	4			1		3			6
9	9	3	2				7	1	4

Here, the user will be prompted for a command (the main prompt).

```
> Z
```

Please note that you will need a newline, a carat ('>'), and a space before the prompt.

The next action of the program will depend on the user's command. If the user types an invalid command, then the following message will be displayed:

```
ERROR: Invalid command
```

Show Instructions

If the user types '?', then the menu of options will be displayed again. These are the same instructions that are displayed when the program is first run.

Display the Board

If the user types 'D', then the board will be redrawn. This is the same as the drawing of the board when the program is first run.

Save and Quit

If the user types 'Q', then he or she will be prompted for the filename:

```
What file would you like to write your board to: newGame.txt
```

The program will display the following message:

```
Board written successfully
```

Then the program will terminate.

Edit One Square

If the user types 'E', then the program will prompt him for the coordinates and the value for the square to be edited:

```
What are the coordinates of the square: A1
```

If the value is invalid or if the square is already filled, then the program will display one of the following message and return to the main prompt:

```
ERROR: Square 'zz' is invalid
ERROR: Square 'A1' is filled
```

With a valid coordinate, then the program next prompts the user for the value:

```
What is the value at 'A1': 9
```

If the user types a value that is outside the accepted range ($1 \leq \text{value} \leq 9$) or does not follow the rules of Sudoku, then a message appears and the program returns to the main prompt:

```
ERROR: Value '9' in square 'A1' is invalid
```

Show Possible Values

If the user types 's', then the program will prompt him for the coordinates and display the possible values:

```
What are the coordinates of the square: A1
```

The same parsing logic applies here as for the Edit One Square case. Once the user has selected a valid coordinate, then the program will display the possible values:

```
The possible values for 'A1' are: 1, 5, 8, 9
```

After the message appears, the program returns to the main prompt.

Project 11

The first part of the project is the design document. This consists of three parts:

1. Create a structure chart describing the entire Sudoku program.
2. Write the pseudocode for the function `computeValues()`. This function will take as parameters coordinates (row and column) for a given square on the board and calculate the possible values for the square. To do this, `computeValues()` must be aware of all the rules of Sudoku. Make sure to include both the logic for the rules of the game (only one of each number on a row, column, and inside square), but also to display the values.
3. Write the pseudocode for the function `interact()`. This function will prompt the user for his option (ex: 'D' for "Display the board" or 'S' for "Show the possible values for a square") and call the appropriate function to carry out the user's request. Note that the program will continue to play the game until the user has indicated he is finished (with the 'Q' option).

On campus students are required to attach [this rubric](#) to your design document. Please self-grade.

Project 12

The second part of the Sudoku Program project (the first part being the design document due earlier) is to write the code necessary to make the Sudoku appear on the screen:

```
Where is your board located? prj4.txt
Options:
? Show these instructions
D Display the board
E Edit one square
S Show the possible values for a square
Q Save and Quit

  A B C D E F G H I
1  7 2 3 |   | 1 5 9
2  6   | 3  2 |   8
3  8   |   1 |   2
  ----+----+----
4   7  | 6 5 4 |  2
5     4 | 2  7 |  3
6   5  | 9 3 1 |  4
  ----+----+----
7  5   |   7 |   3
8  4   |  1  3 |   6
9  9 3 2 |   | 7 1 4

> E
What are the coordinates of the square: e5
What is the value at 'E5': 8

> q
What file would you like to write your board to: deleteMe.txt
```

Perhaps the easiest way to do this is in a four-step process:

1. Create the framework for the program using stub functions based on the structure chart from your design document.
2. Write each function. Test them individually before "hooking them up" to the rest of the program.
3. Verify your solution with testBed:

```
testBed cs124/project12 project12.cpp
```

4. Submit it with "Project 12, Sudoku" in the program header.

An executable version of the project is available at:

```
/home/cs124/projects/prj12.out
```

Project 13

The final part of the Sudoku Program project is to enforce the rules of Sudoku. This means that there can be only one instance of a given number on a row, column, or inside square.

```
Where is your board located? prj4.txt
Options:
? Show these instructions
D Display the board
E Edit one square
S Show the possible values for a square
Q Save and Quit

  A B C D E F G H I
1  7 2 3|   | 1 5 9
2  6   |3  2|   8
3  8   |  1 |   2
  ----+----+----
4   7  |6 5 4|  2
5     4|2  7|3
6   5  |9 3 1|  4
  ----+----+----
7  5   |  7 |   3
8  4   |1  3|   6
9  9 3 2|   |7 1 4

> s
What are the coordinates of the square: b2
The possible values for 'B2' are: 1, 4, 9

> e
What are the coordinates of the square: b2
What is the value at 'B2': 2
ERROR: Value '2' in square 'B2' is invalid

> q
What file would you like to write your board to: deleteMe.txt
```

Perhaps the easiest way to do this is in a four-step process:

1. Start with the code written in Project 12.
2. Fix any necessary bugs.
3. Verify your solution with testBed:

```
testBed cs124/project13 project13.cpp
```

4. Submit it with "Project 13, Sudoku" in the program header.

An executable version of the project is available at:

```
/home/cs124/projects/prj13.out
```